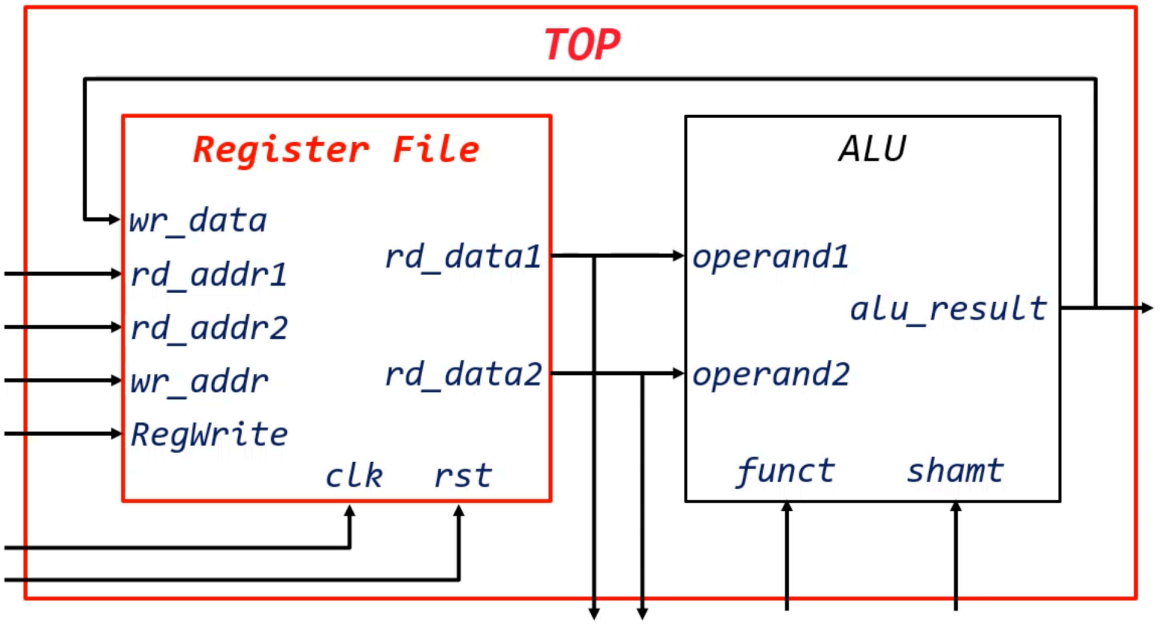
Lab 2 Report

과제 개요 및 구현 과정

이번 과제는 Verilog를 사용하여 asynchronous read와 synchronous write를 지원하는 레지스터 파일을 구현하는 과제다. Skeleton code의 `ALU.v`, `TOP.v`, `RF.v` 파일의 내용을 수정하였다.



전체적인 구조는 다음과 같다. TOP 모듈 내의 서브 모듈 RF와 ALU가 있고, RF에서 값을 읽어와 ALU에서 연산을 실행한다.

```
case (funct)
    `ALU_ADDU: alu_result = operand1 + operand2;
    `ALU_AND : alu_result = operand1 & operand2;
    `ALU_NOR : alu_result = ~(operand1 | operand2);
    `ALU_OR  : alu_result = operand1 | operand2;
    `ALU_SLL : alu_result = operand2 << shamt;
    `ALU_SRA : alu_result = $signed(operand2) >>> shamt;
    `ALU_SRL : alu_result = operand2 >> shamt;
    `ALU_SUBU: alu_result = operand1 - operand2;
    `ALU_XOR : alu_result = operand1 ^ operand2;
    `ALU_SLT : alu_result = ($signed(operand1) < $signed(operand2)) ? 32'd1 : 32'd0; // Signed 비교
    `ALU_SLTU: alu_result = (operand1 < operand2) ? 32'd1 : 32'd0; // Unsigned 비교
    `ALU_EQ  : alu_result = (operand1 == operand2) ? 32'd1 : 32'd0;
    `ALU_NEQ : alu_result = (operand1 != operand2) ? 32'd1 : 32'd0;
    `ALU_LUI : alu_result = operand2 << 16; // LSB 16-bit에 있는 Immediate 값을 MSB 16-bit로 이동
endcase
```

`ALU.v` 파일은 Lab 1 과제 그대로 구현하였다.

```
    // FIXME
    // Instantiate modules and connect them!
    RF rf (.clk(clk), .rst(rst),
    .rd_addr1(rd_addr1), .rd_addr2(rd_addr2),
    .rd_data1(rd_data1), .rd_data2(rd_data2),
    .RegWrite(RegWrite), .wr_addr(wr_addr), .wr_data(wr_data));
    ALU alu (.operand1(rd_data1), .operand2(rd_data2), .shamt(shamt), .funct(funct), .alu_result(wr_data));
```

`TOP.v` 파일에서 RF와 ALU 모듈을 instantiate 하고 연결해주었다.

1. RF 모듈

- 1-bit clock `clk`과 synchronous reset signal `rst`를 TOP 모듈 내의 동명의 clock `clk`, reset signal `rst`와 연결하였다.
- 2개의 5-bit read input port `rd_addr1`과 `rd_addr2`를 TOP 모듈 내의 동명의 read input port `rd_addr1`과 `rd_addr2`와 연결하였다.
- 2개의 32-bit read output port `rd_data1`과 `rd_data2`를 TOP 모듈 내의 동명의 read output port `rd_data1`과 `rd_data2`와 연결하였다.
- 3개의 write 관련 port `RegWrite`, `wr_addr`, `wr_data`를 각각 TOP 모듈의 동명의 변수와 연결하였다.

2. ALU 모듈

- ALU 모듈의 2개의 32-bit 피연산자 `operand1`, `operand2`를 RF 모듈에서 output으로 출력하여 TOP 모듈의 `rd_data1`, `rd_data2`와 연결하였다.
- ALU 모듈의 5-bit shift amount `shamt`, 4-bit 연산자 `funct`를 각각 TOP 모듈의 입력으로 받는 `shamt`, `funct`와 연결하였다.
- ALU 모듈의 32-bit output register `alu_result`와 TOP 모듈의 `alu_result`와 연결하였다.

`RF.v` 파일에서는 과제의 목표인 레지스터 파일을 구현하였다.

```
    always @(*) begin
        rd_data1 = register_file[rd_addr1];
        rd_data2 = register_file[rd_addr2];
```

먼저 Asynchronous read는 `rd_addr1`과 `rd_addr2`의 값이 바뀌자마자 `rd_data1`과 `rd_data2`의 값이 바뀌어야 한다. 따라서 `clk`의 positive edge와 관계없이 모든 상황에서 `rd_data`의 값을 레지스터 파일 `register_file`의 read address `rd_addr`에서 읽어온다.

```
    always @(posedge clk) begin
        if (rst) begin
            $readmemh("initial_reg.mem", register_file);
        end
        else if (RegWrite) begin
            register_file[wr_addr] <= wr_data;
        end
    end
```

Synchronous write는 clock의 positive edge가 될 때 `wr_data`가 쓰여진다. Reset signal일 경우를 제외하고 positive edge `posedge`가 될 때 write enable signal `RegWrite`가 설정되었다면 레지스터 파일 `register_file`의 write address `wr_addr`에 write data `wr_data`를 쓴다.

테스트

```
HW02 Skeleton Code  iverilog ALU.v RF.v TOP.v TOP_tb.v
HW02 Skeleton Code  ./a.out
```

주어진 테스트벤치와 구현한 ALU, RF, TOP 모듈을 컴파일하고 실행하였다.

```
PASSED
PASSED
PASSED
PASSED
PASSED = 10000, FAILED = 0
TOP_tb.v:124: $finish called at 100035000 (1ps)
```

총 10,000개의 테스트 모두 **PASSED**로 통과하였다.

과제를 하면서 어려웠던 부분

`TOP.v` 파일에서 RF 모듈과 ALU 모듈을 TOP 모듈에 연결하는 과정에서 전체적인 HDL을 구상하는데 변수명이 비슷하여 헷갈리는 부분이 있었다. HDL을 구상하고 bit 수를 비교하여 연결을 완성할 수 있었다.